Lecture 33: Vision-Language Models I

How a model sees

The most ordinary thing you do with a chat model

You paste a photo of a handwritten receipt and ask what the total is. It answers correctly. You paste a chart and ask which quarter dropped. It answers.

Nothing in this course so far can do that.

A transformer (ch9) consumes a **sequence of vectors**. Text arrives as one naturally: split into tokens, look each up in an embedding table.

An image is a **grid of pixels**. There is no sentence, no vocabulary, and no obvious order. Nothing to look up.

This lecture is that bridge. The next one is the harder direction: making the same model **draw**.

Two questions that sound the same and are not

1. How does a model see an image?

Settled. One recipe, essentially everyone uses it, and the arguments left are about efficiency.

This lecture. You will be able to explain every open-weights vision-language model released in the last three years.

2. How does a model draw an image?

Not settled. Three competing architectures, actively argued about, and the leading products do not agree with each other.

L34. The disagreement is the interesting part, and it comes from a genuine conflict with what you built in ch10.

A **vision-language model** (**VLM**) is any model that takes both images and text as input. The good ones do both questions at once.

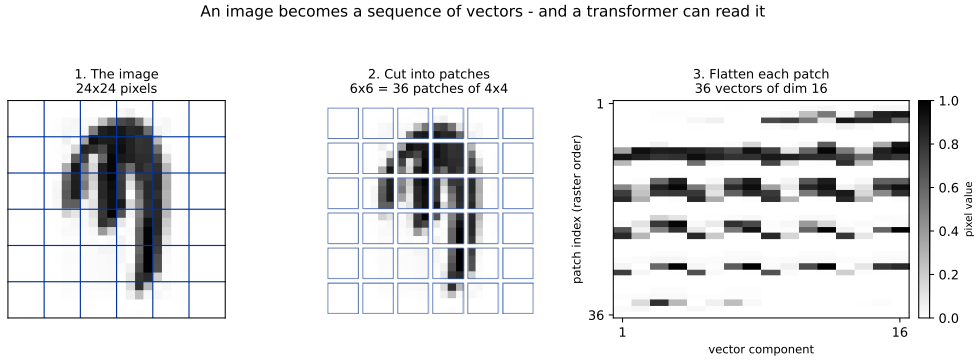
Outline

An image is not a sequence

So make it one. The trick is almost insultingly simple.

Patchify

Cut the image into fixed squares. Flatten each square into a vector. Push each vector through **one linear layer**. Done - the image is now a sequence, and every tool from ch9 applies unchanged.



Shown on a letter from ch10, at 24×24 with 4×4 patches. Each patch becomes a 16-dimensional vector; the linear layer maps it to whatever width the transformer wants.

The arithmetic that governs everything downstream

A patch grid of side s/p gives $(s/p)^2$ tokens. This number comes back as a **cost** in every design decision for the rest of the lecture.

Resolution	Patch	Grid	Tokens
224×224	16×16	14×14	196
224×224	14×14	16×16	256
336×336	14×14	24×24	**576**
896×896	14×14	64×64	4096

The highlighted row is **LLaVA-1.5** (Liu et al., 2023), the reference open vision-language model. Remember **576**.

Tokens grow with the *square* of resolution, and attention costs the *square* of tokens. Doubling resolution is roughly $16\times$ the attention work.

The Vision Transformer

Dosovitskiy et al. (2021) put patches into a standard transformer encoder and changed almost nothing else. The **Vision Transformer** (**ViT**) is the result, and it underlies essentially every vision encoder in use today.

1. Patchify, then one linear projection.
2. **Add position embeddings.** Without them a picture is an unordered bag of patches - self-attention has no idea which patch was where.
3. Optionally prepend a **classification token** (**CLS**) whose output represents the whole image.
4. Run N ordinary transformer blocks.

The naming is honest. There is no vision-specific machinery here. It is the ch9 architecture, fed differently.

Predict first

A CNN (ch6) is built around a strong assumption: **nearby pixels belong together**, hard-wired by the convolution kernel. A Vision Transformer has no such wiring - every patch can attend to every other patch from layer one.

So: does a Vision Transformer need convolution to learn that neighbouring patches are related?

Predict first

A CNN (ch6) is built around a strong assumption: **nearby pixels belong together**, hard-wired by the convolution kernel. A Vision Transformer has no such wiring - every patch can attend to every other patch from layer one.

So: does a Vision Transformer need convolution to learn that neighbouring patches are related?

No - given enough data. With large training sets it learns local structure by itself, and beats comparable CNNs.

But on small data it loses. The convolution's assumption is genuinely correct, and a model given the right prior needs fewer examples to find it.

This is the bias-variance tradeoff from ch2, wearing an architecture costume: a helpful constraint is worth a lot when data is scarce and becomes a ceiling when it is not.

Two encoders, one space

Patches are only useful if their meaning lines up with words.

The problem with a plain Vision Transformer

Train a Vision Transformer to classify 1000 categories and its features are excellent **at those 1000 categories**. Ask about anything else and the representation was never asked to encode it.

We want an image representation shaped by **language**, so that "a photo of a dog wearing a hat" and the corresponding picture land in the same place. Then a language model can read the image features because they already speak its language.

The supervision for that is lying around the internet in enormous quantity: images that came with captions.

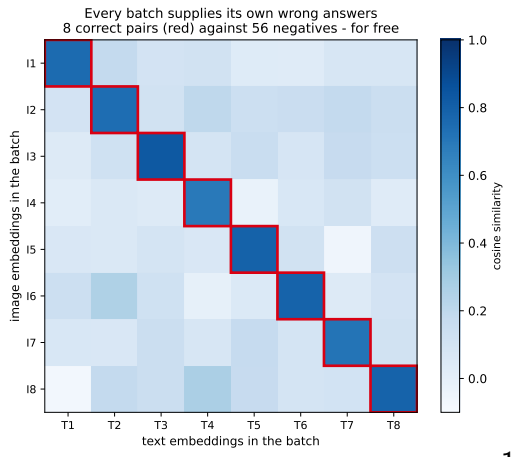
CLIP

Contrastive Language-Image Pre-training (**CLIP**), Radford et al. (2021). Two encoders - one for images, one for text - trained on **400 million** image-caption pairs.

Take a batch of N pairs. Embed all N images and all N captions. Now score every image against every caption:

- N pairs are **correct** matches.
- $N^2 - N$ pairs are **wrong**.

Train so the correct ones score high and the rest score low. That is the entire objective.



Illustrative similarity values, drawn from a specified distribution. The **structure** is exact.

Why the free negatives matter so much

Supervised learning needs someone to write down the right answer. Contrastive learning only needs pairs that **belong together**, and it manufactures its own wrong answers from the rest of the batch.

At $N = 32{,}768$, one training step supplies about **one billion** negative comparisons at the cost of embedding 32,768 items.

So batch size is not just a speed knob here. It changes how hard the task is, and therefore what the model learns. Small batch, easy task, weaker features.

SigLIP (Zhai et al., 2023) - **Sigmoid Loss for Language-Image Pre-training** - replaces the batch-wide softmax with an independent sigmoid on each pair. No normalisation across the batch, so training works at smaller batches and scales more easily. It is now a common default encoder in open vision-language models.

Zero-shot classification falls out for free

CLIP was never trained to classify anything. But it can, with no classifier head and no fine-tuning:

1. Write each class as a sentence: *"a photo of a cat"*, *"a photo of a dog"*, *"a photo of a marshrutka"*.
2. Embed all of them with the text encoder.
3. Embed the image. Pick the sentence with the highest similarity.

The class list is now a **string**, decided at inference time. Adding a category costs a sentence, not a retraining run.

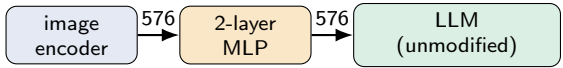
Bounded by the captions it saw. Concepts rare on the web - specialist medical images, minority scripts - are weak. "Zero-shot" is not "knows everything".

Three ways to give an LLM eyes

We have image tokens that mean something. Now attach them to a language model.

Design A - a projector

The simplest thing that works, and it works remarkably well. **LLaVA** (Liu et al., 2023).



A **multi-layer perceptron** (**MLP**) maps each of the 576 visual vectors into the language model's embedding space. They are then simply **prepended to the text tokens**, exactly as if they were words.

The language model is not modified at all. No new layers, no architecture change. That is why this took off - anyone could bolt vision onto any open LLM.

Cost: all 576 tokens sit in the context window for the whole conversation.

Design B - a resampler

If 576 tokens is too many, **learn to compress them** to a fixed budget. **Flamingo** (Alayrac et al., 2022) and **BLIP-2** (Li et al., 2023).

A small set of **learned query vectors** cross-attends to the image features and outputs one vector each. The output count is **constant** no matter how many patches went in - the image is summarised, not passed through.

Resolution and token cost are now **decoupled**. Feed a bigger image, pay the same downstream price. This is what makes video remotely affordable.

A fixed budget is a fixed budget. Whatever those queries fail to pick up is gone. Dense text in an image suffers badly.

A detail worth stealing. Flamingo injects vision through gated cross-attention whose gate is initialised to **zero**. At step one the vision path contributes exactly nothing, so the model *is* the original language model, and vision fades in as the gate opens. Adding a modality can therefore never damage what already worked - the same idea as a residual branch starting at zero.

Design C - early fusion

Remove the bridge entirely. **One transformer**, both modalities, from the first layer.
Chameleon (Meta, 2024) is the clearest example, and L34 is mostly about this family.

There is no adapter to train, no frozen encoder, no projection. Images and text are just two kinds of token in one vocabulary, and one loss covers both.

This is the only design that can **generate** images as naturally as it reads them. A projector is a one-way street.

Costs: you cannot reuse a pretrained language model as-is, training is much more expensive, and mixed-modality training is unstable enough that Chameleon needed specific fixes to converge at all.

Which won

	A: projector	B: resampler	C: early fusion
Token cost	full (576)	fixed, small	full
Pretrained LLM reusable	yes	yes	no
Training cost	lowest	low	highest
Can generate images	no	no	**yes**
Detail preserved	high	lossy	high

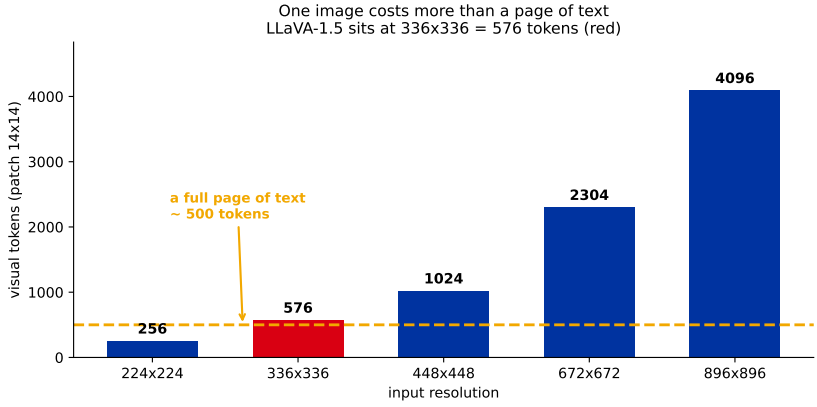
It depends on whether the model must also draw. For understanding alone, **A still dominates** - Qwen2-VL, InternVL and the LLaVA family all keep a separate encoder and a projector. **C is what you need the moment the model must generate images**, which is why every unified model in L34 is early-fusion. B survives where the token budget binds, above all in video.

So if you are building something today with an open model and a modest budget, A is the right default - and that is not a compromise, it is the current state of the art for reading images.

What it costs, and what breaks

Every failure in this section traces back to one number.

An image is expensive



A 336×336 thumbnail costs more context than a full page of writing. A phone photo at native resolution would cost more than this entire lecture as text.

Stop resizing everything to a square

The fixed-resolution habit is inherited from image classification, and it is actively harmful for documents: a wide receipt squashed to 336×336 has unreadable text **before the model sees it**.

Naive Dynamic Resolution (Qwen2-VL, Wang et al., 2024): keep the original aspect ratio, emit however many tokens the image actually needs. A small icon costs few tokens; a dense page costs many.

This single change is most of why recent models became usable for **screenshots, documents and charts**, which is now the dominant real-world use.

The active research direction is the other side of the same coin: **token pruning and merging** - drop or combine patches that carry little information, since most of a photograph is background.

Where vision-language models still fail

These are not random weaknesses. Each has a cause you can now name.

Failure	**Why**
Counting objects	Attention pools information; nothing in the training objective ever rewarded keeping a tally.
Precise spatial relations	Position embeddings give rough layout, not geometry.
Reading clocks and gauges	Needs exact angles from a coarse patch grid.
Dense or small text	Below the patch resolution the characters are simply not there.

Notice the pattern: most of these are the **token budget** again. The information was destroyed at the encoder, and no amount of language-model capability downstream can restore it. Same lesson as ch10, where halving a 1-2 pixel stroke twice erased it before the deep layers ever saw it.

Hallucination, in the visual case

A vision-language model will confidently describe objects that are not in the picture - most often objects that *usually* appear alongside what is there.

Ask about a kitchen photo and you may be told about a refrigerator that is not in frame. Kitchens have refrigerators, and the language model knows that far more strongly than it trusts 576 blurry vectors.

The language prior overrides the image. It is the *language* model that was trained on trillions of tokens; the visual evidence is comparatively thin and easy to outvote.

Practical consequence: a fluent, specific description is not evidence that the model looked carefully. If the answer matters, ask about what is *absent*, or ask twice with the object named differently.

Recap

1. Cut an image into patches, project each one, and it is a sequence a transformer can read. That is the whole bridge.
2. A Vision Transformer is the ch9 architecture, fed differently. Position embeddings are what stop it being a bag of patches.
3. CLIP aligns images and text by contrast, using the batch itself as negatives. Zero-shot classification falls out for free.
4. Attach it to a language model with a projector, a resampler, or no bridge at all. For reading images the projector still wins; the bridge only disappears when the model must also draw.
5. One 336×336 image costs **576** tokens, and most failures trace back to that number.

Next: reading an image is settled. **Drawing** one from the same model is not.

L34 starts from a real conflict: your ch10 diffusion model works in continuous space, and a language model emits discrete tokens. Three different architectures resolve that contradiction three different ways, and we will measure what the most popular one costs on our own letters.